Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Course Project

Team leader: Hongyu Guo
Team members: Hongyu Guo (individual project)

Abstract

Can a classifier identify which large language model wrote a response? We study six model families in TXD-22 using 17,988 responses and a split that keeps answers to the same prompt together. A CNN and a bidirectional LSTM trained from scratch reach 87.5% and 84.6% test accuracy, respectively; TF-IDF logistic regression performs best at 92.5% (chance: 16.7%). Prompt-only classification is near chance, and adding the prompt produces no consistent improvement. Transfer between two question forms produces small observed drops. Style features and retraining ablations show that substantial signal survives punctuation removal and typography normalisation, although the results are limited to this dataset and its collection process.

1 Introduction

Different LLMs may leave recognisable patterns in their word choice and response structure. We treat model identification as six-way classification and organise the experiments around four research questions:

RQ1: Can we identify which LLM generated a response?
RQ2: Does the user's prompt help identify the LLM?
RQ3: Do LLM fingerprints generalise across question types?
RQ4: What characteristics distinguish different LLMs?

Our main finding is that the responses are distinguishable within TXD-22, but a simple baseline outperforms both neural networks. Audits and ablations help explain the signal and the limits of that finding.

2 Problem Definition and Dataset Curation

2.1 Dataset introduction

Each example is a triple (LLM name, input prompt, output response). For RQ1 the classifier receives only the response and predicts its model-family label. We use TXD-22 (A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification), released by Iqbal and Kashem on Mendeley Data under CC BY 4.0 [1]; we did not generate responses ourselves. According to the dataset description, the authors sent questions spanning 30 academic and professional subjects to 17 AI systems between January and March 2025, requesting answers of at most about 300 words. They also collected human and mixed-source text and describe normalising Unicode and whitespace. The released CSV contains 66,000 rows across 22 classes, with columns `Question`, `Answer` and `Source`. Model versions, generation settings and API versus chat-interface access are undocumented.

2.2 Model-family selection and cleaning

We select ChatGPT, Gemini, Claude, Llama, DeepSeek and Qwen, six model-family labels from different developers. Human and mixed-authorship classes are excluded; restricting the study to these six also avoids labels for products that may serve multiple underlying models. Filtering occurs

before vocabulary construction or training. The subset contains academic questions and answers, with no private information identified in our review. We read the CSV as Windows-1252, remove one spreadsheet-error response and 11 exact within-model duplicates, and retain 17,988 examples. There are no missing or empty responses and no exact response shared across model labels. Case, punctuation and symbols are preserved because they may carry identifying information.

Table 1: Dataset after filtering and cleaning. Words and tokens are mean counts per response; median and P99 refer to token counts. The classes remain approximately balanced.

Model	Raw	Removed	Kept	Train	Val	Test	Words	Tokens	Median	P99
ChatGPT	3000	0	3000	2100	450	450	245	291	286	479
Gemini	3000	8	2992	2093	450	449	266	322	355	579
Claude	3000	0	3000	2100	450	450	298	344	337	538
Llama	3000	1	2999	2099	449	451	210	244	240	351
DeepSeek	3000	3	2997	2098	450	449	267	336	336	470
Qwen	3000	0	3000	2100	450	450	219	266	271	395
Total	18000	12	17988	12590	2699	2699				

2.3 Audit and evaluation split

The same questions appear across model labels: 3,082 raw strings reduce to 3,000 groups after normalising case, punctuation and non-ASCII characters. Of these, 99.1% have responses from all six models. A row-level split could therefore test on questions already represented in training. We instead assign every response to a question to the same split, using `StratifiedGroupKFold` with 20 folds: 14 for training, three for validation and three for testing. This yields a 70/15/15 split with 12,590, 2,699 and 2,699 rows, respectively, and 0 shared prompt groups. The assignment is saved and reused in every experiment. Hyperparameter and checkpoint selection use validation data, with final scores measured on the held-out test split.

The audit revealed an encoding artefact: apostrophes in Qwen prompts can differ from the garbled characters in the same questions under other labels. Consequently, 65 raw prompt strings occur only under Qwen. Grouping on normalised text prevents those versions of the same question from crossing splits; RQ2 separately tests whether raw prompt typography leaks the label.

Response length is a possible cue: Llama averages 244 tokens and Claude 344. Self-identification is rare, with at most 0.07% of a model's responses mentioning its own family. Markdown is also scarce: at most 0.9% contain newlines and 2.0% contain bold markup. Curly apostrophes differ substantially across models, occurring in 47.1% of ChatGPT and 52.2% of DeepSeek responses. Such typography may reflect either generation or collection, so we test it by retraining after normalisation. Detailed audit tables and the length-distribution plot remain available in the repository.

3 Models and Training

3.1 Representations and architectures

The neural models use a case-sensitive word-level tokenizer that emits letter runs, digit runs, newlines and individual punctuation or symbol characters. For example, `don't` becomes three tokens. The vocabulary is built only from training data: tokens occurring at least 2 times are retained up to 30,000, giving 30,005 entries including special tokens. Unseen tokens map to an unknown token. Responses are limited to 512 tokens, which fully covers 99.2% of training examples. Both networks learn randomly initialised 200-dimensional embeddings without pretraining.

CNN. Following Kim [2], three parallel convolutions use kernel sizes 3, 4 and 5, with 256 filters each. ReLU and max-pooling over time produce a concatenated 768-dimensional vector, followed by dropout (0.3) and a linear classifier with six outputs. The model has 6,620,782 parameters. Its filters detect local token patterns anywhere in the response.

BiLSTM. The recurrent network uses a bidirectional LSTM [3] with 2 layers and hidden size 256 per direction. Packed sequences skip padding. Max- and mean-pooling over the real token outputs give a 1024-dimensional vector, followed by dropout (0.5) and a six-output linear layer. Embedding dropout is also applied. The model has 8,522,094 parameters. Pooling across time allows evidence anywhere in a response to contribute.

3.2 Training and model selection

Both models minimise cross-entropy using AdamW. Table 2 summarises the shared settings. Batches group similarly sized responses to reduce padding, with examples and batch order shuffled. Python, NumPy and PyTorch seeds are fixed and cuDNN uses deterministic mode.

Table 2: Shared neural training settings. Dropout and architecture choices are described above.

Setting	Value
Learning rate / weight decay	0.001 / 0.01
Batch size	64
LSTM gradient clipping	Norm 1
Checkpoint selection	Highest validation macro-F1
Early stopping	3 epochs without improvement
Maximum training length	15 epochs

A validation-only grid with seed 42 tests CNN filters {128, 256} and dropout {0.3, 0.5}, and BiLSTM hidden sizes {128, 256} and layer counts {1, 2}. The selected architectures are those described above and are reused in later experiments. The full grid is saved in `results/tuning/`. RQ1 averages 3 seeds; RQ2–RQ4 use seed 42. We report accuracy and macro-F1, with per-class F1 and confusion matrices to show differences between labels. Reported standard deviations describe seed variability, not confidence intervals. All 57 training runs took 115 minutes on a NVIDIA GeForce RTX 4060 Laptop GPU.

TF-IDF reference. Logistic regression uses case-sensitive word 1–2-grams and character 2–5-grams, sublinear term frequency and 400,000 features. Its regularisation strength ($C = 100$) is selected on validation data. This baseline tests whether frequency-based features are sufficient without learning a neural representation.

4 Research Questions and Results

4.1 RQ1: Can we identify which LLM generated a response?

Main finding: Both neural models identify responses well above chance; TF-IDF achieves the highest accuracy at 92.5%.

Table 3 shows that both networks substantially exceed chance on held-out prompt groups. The CNN achieves 87.5 ± 0.8% accuracy and the BiLSTM 84.6 ± 1.4%. The CNN has a higher average score and trains faster: the seed-42 runs take 30 and 266 seconds, respectively. With only 3 seeds, the architectural comparison remains limited.

TF-IDF logistic regression performs best at 92.5%. A plausible explanation is that the networks must learn millions of parameters from 12,590 training examples, while the baseline directly represents recurring word and character patterns. The CNN's training accuracy is 93.9% at its se-

Table 3: RQ1 on 2,699 test responses. Neural results are mean ± standard deviation over 3 seeds. The final six columns are per-class F1 (%).

Model	Accuracy	Macro-F1	ChatGPT	Gemini	Claude	Llama	DeepSeek	Qwen
CNN	87.5 ± 0.8	87.5 ± 0.8	80.5	88.0	96.6	95.4	82.4	82.0
BiLSTM	84.6 ± 1.4	84.4 ± 1.6	76.8	82.2	96.1	92.3	79.7	79.4
TF-IDF + LR	92.5	92.4	88.2	92.8	98.0	98.9	89.2	87.4
Chance	16.7	–	–	–	–	–	–	–

lected epoch versus 87.3% on validation, consistent with overfitting. Training curves are available in `figures/training_curves.pdf`. These observations help explain the baseline's advantage but do not establish a single cause.

Claude and Llama are easiest to recognise: CNN F1 is 96.6% and 95.4%, respectively. ChatGPT and Qwen are harder, at 80.5% and 82.0%. Figure 1 shows that these labels are confused in both directions. Claude's long sentences and Llama's shorter, less diverse responses provide possible explanations for their distinctive profiles; RQ4 examines these characteristics.

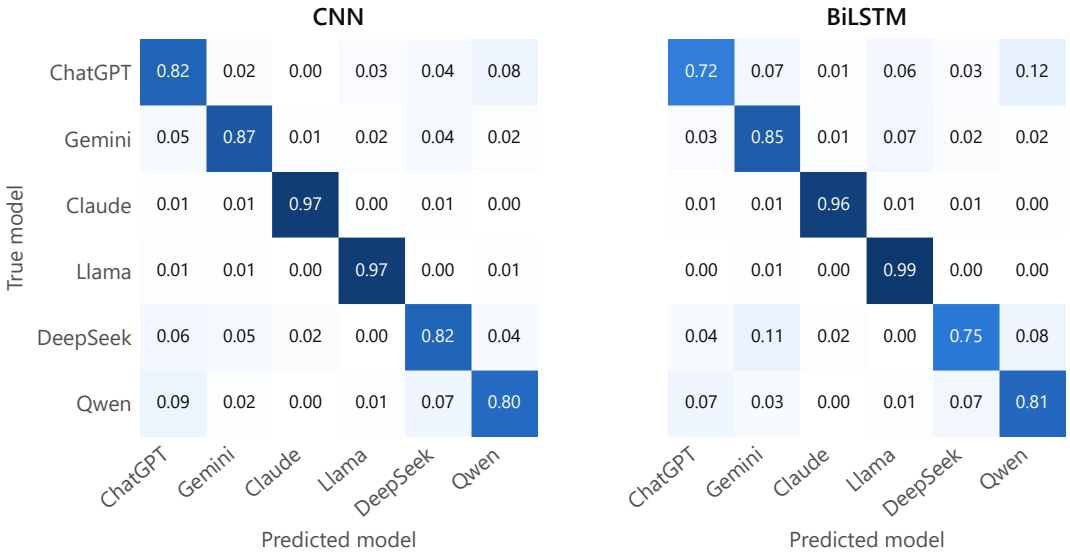


Figure 1: RQ1 confusion matrices averaged over 3 seeds. Rows are true labels, columns are predictions, and each row sums to one. The diagonal gives per-class recall.

4.2 RQ2: Does the user's prompt help identify the LLM?

Main finding: Prompt-only accuracy is near chance, and adding the prompt gives no consistent improvement in these runs.

We compare prompt-only, response-only and combined inputs. Combined inputs concatenate `[INP] prompt [OUT] response`, allowing 128 prompt tokens and 512 response tokens; the longest prompt has only 22 tokens. All comparisons in Table 4 use seed 42. Thus its response-only scores (CNN 86.6%, BiLSTM 83.3%) differ from the multi-seed means in RQ1.

Prompt-only prediction. Accuracy is near 16.7%, as expected when the same question appears under all six labels. Small deviations arise from incomplete groups and prompt encoding differences. Among 57 test rows whose raw prompt is not shared by all six models, the baseline correctly identifies all 10 Qwen rows and the CNN identifies 9. Removing the encoding cue yields prompt-only accuracies of 16.5%, 16.7% and 16.7%. Prompt-only success can therefore reflect dataset assembly rather than a

Table 4: RQ2 test accuracy and macro-F1 (%), seed 42. "ASCII" removes non-ASCII characters and apostrophes from prompts before retraining. Chance refers to accuracy.

	Input only		Input only (ASCII)		Output only		Input + output	
Model	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
CNN	16.8	12.6	16.5	10.9	86.6	86.6	87.5	87.5
BiLSTM	16.7	9.9	16.7	10.4	83.3	83.0	82.3	82.3
TF-IDF + LR	16.9	14.0	16.7	13.9	92.5	92.4	92.6	92.6
Chance	16.7	–	16.7	–	16.7	–	16.7	–

property of the generating model.

Adding the prompt. Compared with response-only input, accuracy changes by +0.9 points for the CNN, −1.1 for the BiLSTM and +0.1 for TF-IDF. These runs show no consistent benefit. Because they use one seed, small differences should be interpreted cautiously; they do not establish that prompt context could never help under a different dataset design.

4.3 RQ3: Do LLM fingerprints generalise across question types?

Main finding: The observed neural accuracy drops are only 0.3–1.5 percentage points under this limited question-form shift.

TXD-22 has no task labels or clear coding, calculation and creative-writing subsets. We therefore test a narrower shift: *definitional* prompts beginning with words such as "what" or "define", versus *explanatory* prompts beginning with "explain", "how" or "why". Definitional openings that also contain explanatory terms are labelled mixed and excluded, along with other unmatched prompts. Rules are in `src/prepare.py`; a spot-check of ten prompts per type confirmed the intended rule assignments, although some definitional questions ask for more than a definition.

We train in both directions. Definitional training groups are subsampled to match the explanatory group count, giving about 3,478 rows per training set. Early stopping uses only source-type validation rows. For each target test set, we compare same-type and other-type training on the same rows (Table 5). Architecture settings are inherited from RQ1 rather than independently retuned for transfer, so this is not a completely target-independent model-selection protocol.

Table 5: RQ3, seed 42. "In" means same-type training and "Cross" means size-matched other-type training. Drops are percentage-point differences; F1 is macro-F1.

Model	Test question type	n	In acc.	Cross acc.	Drop	In F1	Cross F1	Drop
CNN	Definitional	1981	81.8	80.4	+1.5	81.7	80.3	+1.4
CNN	Explanatory	586	83.6	83.3	+0.3	83.6	83.3	+0.3
BiLSTM	Definitional	1981	76.4	75.9	+0.6	76.3	75.7	+0.5
BiLSTM	Explanatory	586	80.7	80.2	+0.5	80.7	80.5	+0.2
TF-IDF + LR	Definitional	1981	89.0	88.3	+0.7	88.9	88.2	+0.7
TF-IDF + LR	Explanatory	586	92.2	91.0	+1.2	92.1	90.9	+1.2

The neural accuracy drops range from 0.3 to 1.5 points. Claude and Llama remain the most identifiable labels, with cross-type F1 at least 88.8% across the four neural settings. These are small observed drops under a limited question-form shift, not evidence of generalisation to substantially different tasks. One seed cannot establish equivalence or rule out a meaningful transfer loss. Both types still concern academic subjects and share the same collection instructions. Training-set size also matters: using all 8,590 definitional rows raises CNN accuracy on explanatory questions from 83.3% to

88.9%. Detailed per-class changes are saved in `results/rq3/transfer.csv`.

4.4 RQ4: What characteristics distinguish different LLMs?

Main finding: Aggregate style statistics are informative, and substantial accuracy survives punctuation removal; truncating responses produces larger drops.

We compute 38 features in five groups: length, lexical diversity and related statistics, punctuation, typography, and formatting. Moving-average type-token ratio (MATTR) uses a fixed 50-word window to reduce the length dependence of ordinary type-token ratio [5]. Logistic regression using these features reaches 72.3% accuracy; histogram gradient boosting reaches 79.7%. Thus aggregate response statistics carry substantial identifying information.

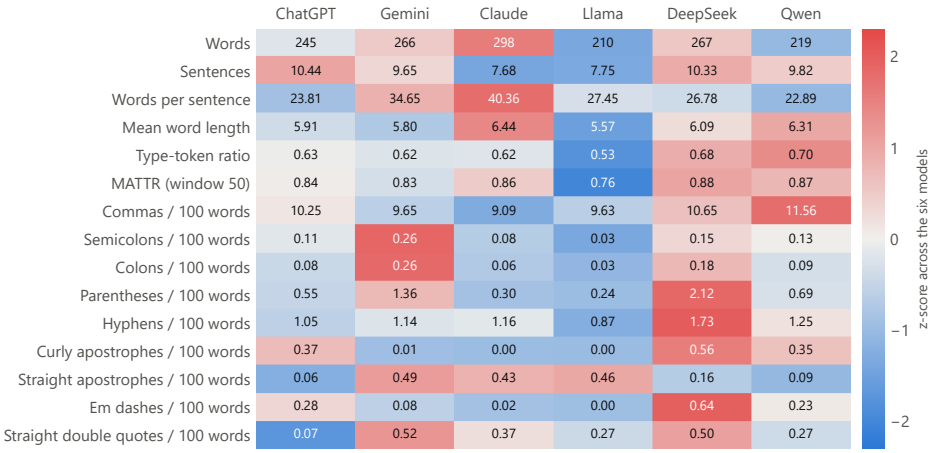


Figure 2: Selected stylometric means by model. Cell values are means; colour gives the z-score of each feature's mean across the six labels. These descriptive differences do not by themselves establish what a neural classifier uses.

Claude averages 40.4 words per sentence, compared with 22.9 for Qwen. Llama has lower MATTR (0.765) than DeepSeek (0.878), and DeepSeek uses more parentheses. Feature-group classifiers reach 59.0% using length alone, 55.2% using lexical statistics, 55.5% using punctuation and 42.4% using typography. Removing length from the full feature classifier lowers accuracy by 4.6 points, suggesting overlapping cues. This does not quantify length's contribution to the neural models.

Training-split n-grams ranked by log-odds with a Dirichlet prior [4] provide examples: Claude favours connecting words such as "while" and "including", DeepSeek favours parentheses and "e.g.", and Qwen favours "for example". Shared prompts reduce topic confounding, but responses may still select different content, so these are descriptive associations rather than pure measures of style. The full n-gram rankings and feature-group results remain in `results/rq4/`.

Ablation design. We apply each transformation to training, validation and test responses, retrain from scratch, and compare with the unmodified seed-42 model. Transformations remove markdown, normalise whitespace, lowercase and remove punctuation, truncate text, mask model/developer names, or normalise typography. Table 6 reports all conditions. The common seed makes runs easier to compare, but single-seed differences have no experiment-specific uncertainty estimate.

Formatting and typography. Markdown removal, whitespace normalisation and name masking alter only 2.5%, 0.6% and 0.8% of responses, respectively, and change accuracy little. Their limited effect is consistent with the audit. Typography normalisation alters 35.7% of responses but also leaves scores close to the originals. This suggests that the tested typographic distinctions are not necessary for high accuracy after retraining, even though they are informative alone. It does not exclude other collection artefacts or prove that the original classifier ignored typography.

Table 6: RQ4 retraining ablations, seed 42. Accuracy is in percent; Δ is the change from the unmodified seed-42 model in percentage points.

	Responses	CNN		BiLSTM		TF-IDF + LR	
Responses modified	changed (%)	Acc.	Δ	Acc.	Δ	Acc.	Δ
Original text	–	86.6	–	83.3	–	92.5	–
(a) Markdown stripped	2.5	87.1	+0.5	82.6	−0.7	92.1	−0.4
(b) Whitespace normalised	0.6	86.8	+0.2	83.1	−0.2	92.6	+0.1
(c) Lowercase, no punctuation	100.0	80.9	−5.7	77.3	−6.0	90.4	−2.1
(d) First 128 tokens	99.1	75.8	−10.8	73.4	−9.9	81.2	−11.3
(d) First 64 tokens	100.0	64.4	−22.3	60.8	−22.5	68.3	−24.2
(e) Model names masked	0.8	85.5	−1.1	83.4	+0.1	92.4	−0.1
(f) Typography normalised	35.7	87.3	+0.7	82.3	−1.0	91.9	−0.6
(c)+(d) Words only, first 64	100.0	58.0	−28.6	56.4	−26.9	64.7	−27.8

Punctuation and amount of text. Lowercasing and punctuation removal reduce CNN accuracy from 86.6% to 80.9% and BiLSTM accuracy from 83.3% to 77.3%. Substantial signal survives, but this condition retains length and word order and does not isolate lexical information. Truncation produces larger drops: the CNN reaches 75.8% with 128 tokens and 64.4% with 64. It removes both response-length information and much of the available text; these effects cannot be separated here. The combined lowercase/no-punctuation, first-64-token condition still reaches 58.0%, above chance. Together, these results support contributions from word patterns and structure without assigning a precise share to each.

4.5 Error analysis

The seed-42 CNN makes 361 errors and the BiLSTM 450 on 2,699 responses; 216 errors are shared. CNN errors are shorter on average than correct predictions (275 versus 304 tokens), consistent with less text providing less evidence, though this association is not causal proof. Two examples, selected as the first test row for their shared confusion pair, illustrate ambiguous responses. Both networks label a Qwen answer about tries as DeepSeek; its opening, "A trie (pronounced 'try')", contains a parenthetical aside associated with DeepSeek in the feature analysis. Both also label a ChatGPT answer about sockets as Qwen; the opening is a generic definition. These examples suggest possible explanations, but examining an opening alone cannot establish why the networks made the prediction. Complete examples and error summaries are saved in `results/rq1/`.

5 Lessons and Limitations

Lessons and experience. The data audit changed the evaluation design: repeated questions required grouped splitting, sparse markdown limited formatting analyses, and encoding differences required normalised group identity. An initial grouping key treated curly and garbled apostrophes as different questions, allowing versions of the same prompt into different splits. We corrected the key and reran the experiments. The practical lesson is to investigate unexpectedly informative inputs and verify group identity beyond exact strings.

The baseline was valuable because it showed that more complex architectures did not automatically improve accuracy. The typography ablation also distinguished descriptive feature differences from signals necessary after retraining. Saving metrics and generating report numbers from them reduced transcription errors, and grouping batches by length roughly halved BiLSTM epoch time.

Limitations. All results concern one dataset of short academic prompts and mostly single-paragraph answers, with undocumented model versions and generation settings. Model-family labels may capture

both generation behaviour and collection practices. Grouped splitting holds out questions, not entire subjects. RQ3 tests only a mild shift and inherits settings selected using the broader validation set. RQ2–RQ4 use one seed, so small changes remain uncertain. Truncation confounds length with lost evidence, and some BiLSTM runs were still improving at the 15-epoch cap.

6 Conclusion

CNN and BiLSTM classifiers identify the six labels well above chance on held-out questions, while TF-IDF logistic regression gives the strongest performance. Prompt-only inputs are near chance and adding prompts offers no consistent gain. A limited question-form transfer test shows small observed drops. Style statistics and retraining ablations reveal several overlapping cues, with substantial performance surviving punctuation removal and typography normalisation. These findings describe TXD-22; broader model attribution requires stronger task shifts and controlled collection conditions.

Use of AI tools. LLM assistants were used for this project.

References

[1] M. S. Iqbal and M. A. Kashem. *TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification*. Mendeley Data, version 1, 2026. `https://doi.org/10.17632/prcjcggtjf.1`

[2] Y. Kim. Convolutional neural networks for sentence classification. In *Proceedings of EMNLP*, pages 1746–1751, 2014.

[3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] B. L. Monroe, M. P. Colaresi, and K. M. Quinn. Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict. *Political Analysis*, 16(4):372–403, 2008.

[5] M. A. Covington and J. D. McFall. Cutting the Gordian knot: The moving-average type-token ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100, 2010.